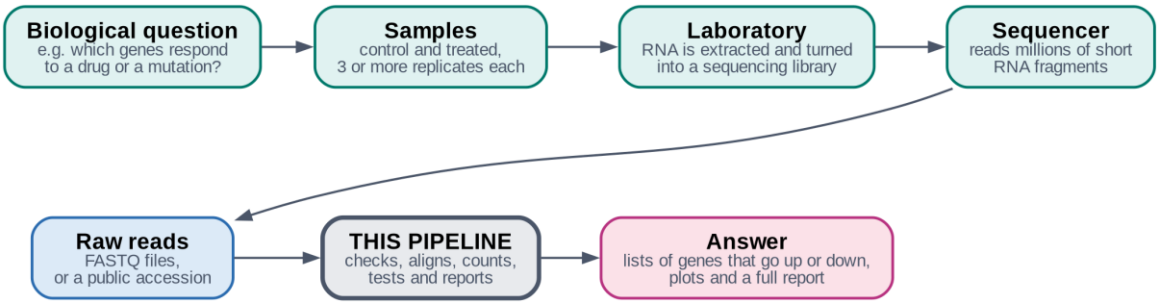
# Bulk RNA-seq Pipeline

## User Guide and Learning Book

From raw sequencing reads to the genes that change: how to install the tool, how each step works, what every tool does, and how to read your results. Written for biologists, students and anyone who wants to use the pipeline without writing code.




| | |
|---|---|
| **Software** | rnaseq-pipeline (command: rnaseq-pipeline) |
| **Version** | 1.1.1 |
| **Guide date** | October 2026 |
| **Source code** | https://github.com/vipin839/rnaseq_pipeline |
| **Developed by** | Vipin |
| **Contact** | www.vipin.social · sainivipin839@gmail.com |
| **Licence** | MIT (free to use, change and share) |

# Contents


Part I · Understanding ..... 3

1. About this guide ..... 3

2. RNA-seq in plain words ..... 4

3. What this pipeline does ..... 6

Part II · Getting started ..... 8

4. What you need ..... 8

5. Download and set up ..... 9

6. Starting the program ..... 12

Part III · The workflow, step by step ..... 14

7. The workflow at a glance ..... 14

8. Steps 1–3 · Getting and checking the data ..... 16

9. Steps 4–6 · Quality control and trimming ..... 18

10. Steps 7–10 · Mapping reads to the genome ..... 21

11. Steps 11–13 · Counting ..... 24

12. Steps 14–17 · Statistics, plots and report ..... 26

13. The decisions you make ..... 30

Part IV · Results ..... 31

14. Where your results are ..... 31

15. Reading your results ..... 33

16. A worked example: Arabidopsis FERONIA ..... 38

Part V · Everyday use ..... 39

17. Resuming, changing and checking a project ..... 39

18. When something goes wrong ..... 41

19. Frequently asked questions ..... 42

Part VI · Reference ..... 43

20. Tools at a glance ..... 43

21. Glossary ..... 45

22. How to cite ..... 47

23. Getting help and licence ..... 48

Appendix · One-page cheat sheet ..... 49

Development and credits ..... 50

# Part I · Understanding

## 1. About this guide

This book explains a complete, ready-made analysis for **bulk RNA sequencing (RNA-seq)** data. You give the pipeline your sequencing reads (or the accession number of a public study) and it gives you back the list of genes whose activity differs between your groups, with plots and a full report.

The guide has three jobs at once:

- **User guide:** how to download, install and run the tool, screen by screen.
- **Help book:** what each message means, what to do when something goes wrong, where every file is.
- **Learning book:** the biology and statistics behind each step, explained in plain words, so that you understand your results and can describe them in a thesis or paper.

### Who this guide is for

Biologists, students and researchers who have RNA-seq data and want reliable results without programming. You do not need to know Python, R or Linux. Where you have to type something, the exact text is given in a grey box, together with what it does.

### How to read it

| If you want to… | Read |
|---|---|
| understand what RNA-seq is | Chapter 2 |
| know what the tool does, in one page | Chapter 3 |
| install it | Chapters 4 and 5 |
| run your first analysis | Chapters 6, 7 and 13 |
| understand one particular step or tool | Chapters 8 to 12, and Chapter 20 |
| understand your results and plots | Chapters 15 and 16 |
| fix a problem | Chapter 18 |
| cite the tool and the methods | Chapter 22 |

### Symbols used in this guide

| Box | Meaning |
|---|---|
| **In plain words** | the idea behind a step, without jargon |
| **Your decision** | a point where the pipeline stops and asks you to choose |
| **What the pipeline checks** | the automatic safety checks of that step |
| **Tip** | a practical hint |
| **Important** | something that can lead to wrong results if ignored |
| Grey box with typewriter text | text you type in the terminal, exactly as shown |

# 2. RNA-seq in plain words

## Genes, RNA and gene expression

Every cell of an organism carries the same DNA, the same set of about 20,000–30,000 genes (in plants and animals). What makes a root cell different from a leaf cell, or a sick cell different from a healthy one, is **which genes are switched on, and how strongly**. When a gene is active, the cell makes copies of it in the form of **messenger RNA (mRNA)**. The more active the gene, the more mRNA copies there are. This activity is called **gene expression**.

> **IN PLAIN WORDS**
>
> DNA is the cookbook; mRNA is the photocopies of the recipes the kitchen is using today. Counting the photocopies tells you which recipes are being cooked, and how often.

## What RNA-seq measures

RNA-seq reads the mRNA of a sample with a DNA sequencer. Because a sequencer can only read short pieces, the RNA is first cut into fragments of a few hundred letters, copied into DNA (cDNA) and tagged with short **adapters** the machine can hold on to. The sequencer then reads one end (**single-end**) or both ends (**paired-end**) of millions of these fragments. Each piece of text it produces is called a **read**.

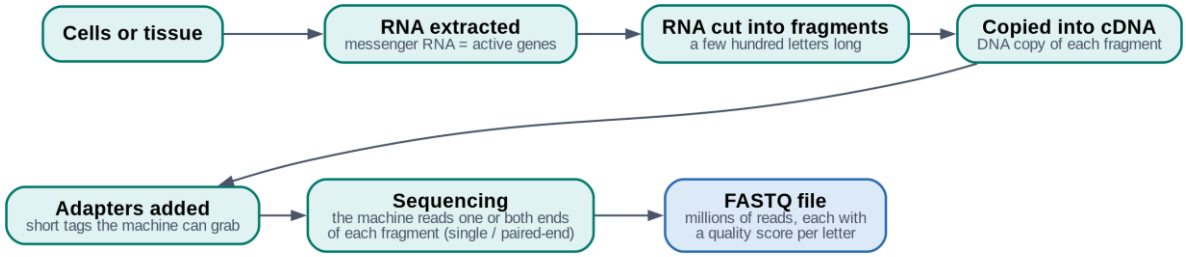



**Figure 1.** What happens in the laboratory before the data reach you.

The reads arrive in **FASTQ files**: for every read, its letters (A, C, G, T) and a quality score for each letter that says how sure the machine was. A typical sample has 10–50 million reads.

## From reads to an answer

A read on its own means nothing. The analysis has to find **where in the genome each read comes from** (alignment), **count how many reads fall on each gene** (quantification), and then **compare the counts between groups** with statistics that take natural variation into account (differential expression).

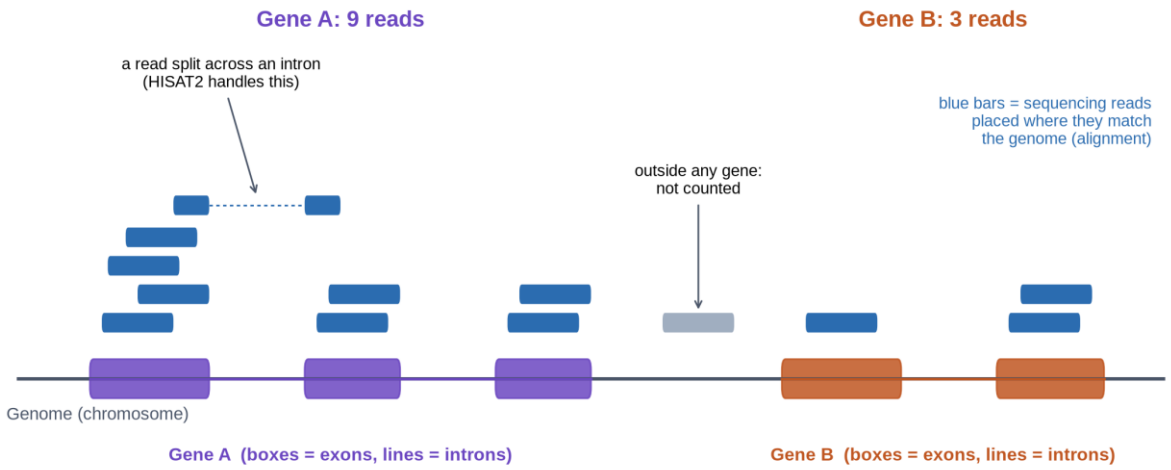



**Figure 2.** Reads are placed on the genome where they match. Reads on a gene's exons are counted for that gene. Here gene A gets 9 reads and gene B gets 3; across many samples these counts are compared.

## Why replicates matter

Two plants, mice or cell cultures are never identical, even under the same conditions. If you compare one control with one treated sample, you cannot tell whether a difference is caused by the treatment or is just natural variation. **Biological replicates** (independent samples of the same group) let the statistics measure that natural variation. This pipeline requires at least 2 replicates per group; 3 or more are strongly recommended.

> **IMPORTANT**
> Technical replicates (the same library sequenced twice, or on several lanes) are **not** biological replicates. The pipeline can add them together, but they do not count towards the replicate number.

## Bulk versus single-cell RNA-seq

**Bulk** RNA-seq measures the average of all cells in a sample (a piece of tissue, a culture). **Single-cell** RNA-seq measures each cell separately and needs different tools. This pipeline is for bulk data only; it recognises single-cell datasets and refuses them, instead of giving wrong results.

## Stranded and unstranded libraries

DNA has two strands, and a gene lies on one of them. Some laboratory kits keep the information about which strand the RNA came from (**stranded** libraries); others lose it (**unstranded**). Knowing this matters for counting, because genes can overlap on opposite strands. The pipeline measures it from your data (step 10) and asks you to confirm.

# 3. What this pipeline does

The pipeline is a single program that runs about 15 specialised, widely used bioinformatics tools in the right order, with the right settings, and checks the result of every one of them. It is interactive: it shows you what it is about to do, explains decisions in plain language, and waits for you when a scientific choice has to be made.

## What you give it

- Your own FASTQ files, **or** an accession number from a public archive (NCBI SRA, NCBI GEO or the European Nucleotide Archive ENA), **or** just keywords to search NCBI for a dataset.
- The organism (the pipeline downloads the matching reference genome and gene annotation).
- Which sample belongs to which group (control, treated, …).

## What you get back

- Lists of **up-regulated** and **down-regulated** genes for every comparison, with fold changes and adjusted p-values.
- Plots: PCA, sample distances and correlations, MA plot, volcano plot and heatmaps (as PNG and PDF).
- A **gene count table** and a transcript abundance table (TPM) you can reuse elsewhere.
- Quality reports for every step (FastQC, MultiQC, fastp, alignment statistics).
- One **HTML report** that brings everything together, and a **manifest** recording versions, settings and file fingerprints, so the analysis can be reproduced.

## The five phases

**Table 1.** The phases of the workflow.

| Phase | Steps | What happens | Main tools |
|---|---|---|---|
| A. Get and check the data | 1–3 | create the project, download or link the reads, verify every read | SRA Toolkit, curl, NCBI E-utilities |
| B. Quality control | 4–6 | measure read quality; trim adapters and poor ends if needed | FastQC, MultiQC, fastp |
| C. Map reads to the genome | 7–10 | prepare the genome; find where each read comes from; check the alignment; find the library strandedness | HISAT2, samtools, RSeQC |
| D. Count | 11–13 | count reads per gene; build the count table; estimate transcript levels | featureCounts, StringTie2 |
| E. Statistics and report | 14–17 | define groups; test every gene; draw plots; write and verify the report | R, DESeq2, ggplot2 |

## What makes this pipeline different

- **It checks everything.** A step is only marked as done when its results have been verified, not when a file simply exists. Numbers are followed through the whole analysis (see "The chain of checks" in Chapter 12).

- **It never guesses scientific decisions.** Trimming, strandedness and the experimental design are always shown to you with an explanation, and you confirm them.
- **You can stop and continue.** Every step saves a checkpoint. After a power cut, a closed laptop or a crash, the analysis continues where it stopped, and nothing half-finished is ever used.
- **Changes are tracked.** If you change a setting, exactly the steps affected by it run again.
- **It records everything** needed to reproduce and cite the analysis.

## What it does not do

- Single-cell RNA-seq (use Cell Ranger, STARsolo or alevin-fry instead).
- Time-course or interaction designs, continuous covariates (planned for later versions).
- Mixing single-end and paired-end samples, or two organisms, in one project (use separate projects).
- Native Windows or macOS (on Windows, use WSL2 with Ubuntu; see Chapter 4).

# Part II · Getting started

## 4. What you need

**Table 2.** Computer requirements.

| Item | Minimum | Comfortable | Why |
|---|---|---|---|
| Operating system | Linux (Ubuntu 22.04 / 24.04), or Windows 10/11 with WSL2 + Ubuntu | same | the scientific tools only exist for Linux |
| Processor | any 64-bit (x86-64) | 8 cores or more | more cores = faster alignment |
| Memory (RAM) | 4 GB for yeast and bacteria; 8 GB for human or mouse | 16 GB or more | the genome index must fit in memory during alignment |
| Disk space | 6 GB for the software | plus about 3× the size of your raw data | reads, trimmed reads, aligned reads and temporary files |
| Internet | needed to install and to download data and genomes | a stable connection | downloads resume automatically if interrupted |
| Administrator rights | not needed | | everything installs in your home folder |

> **TIP**
>
> Put projects on the Linux disk (your home folder), not on a USB stick formatted as FAT32 (4 GB file limit, refused by the pipeline) and preferably not on a Windows drive seen from WSL (`/mnt/c/...`), which is much slower.
>
> The very first human reference takes 1–2 hours to prepare; it is then reused by every project.

### Using Windows? Install Ubuntu first (one time)

Open **PowerShell as Administrator** (Start menu → type PowerShell → right-click → Run as administrator) and type:

```
wsl --install -d Ubuntu-24.04
```

Restart the computer when asked, then open **Ubuntu** from the Start menu and choose a user name and password. Everything in the rest of this guide is typed in that Ubuntu window (the terminal).

# 5. Download and set up

Setting up takes six steps and about 30–60 minutes, most of it waiting for downloads. You do it once. Type each grey box in the Ubuntu terminal and press Enter. Copy-paste works: in the Ubuntu window, right-click (or Ctrl + Shift + V) pastes.

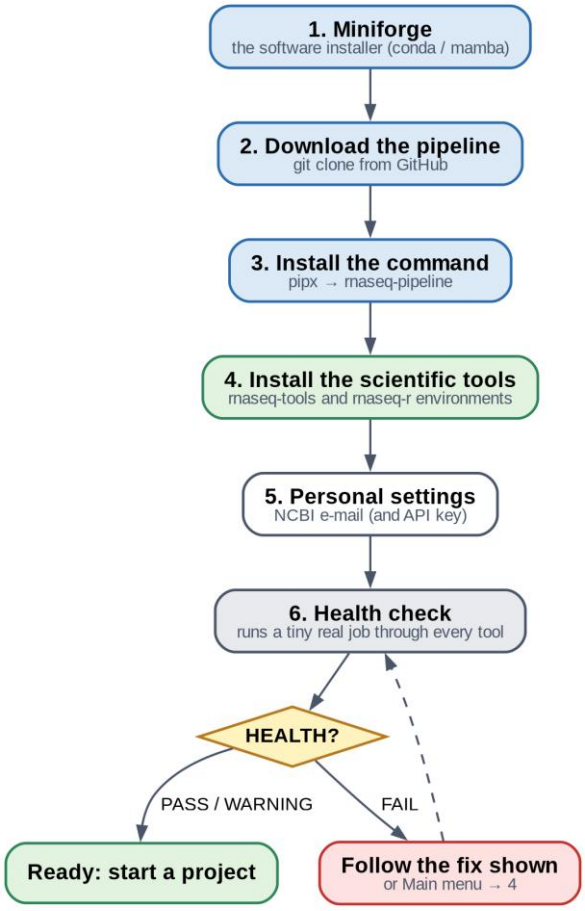



**Figure 3.** The six setup steps.

> **IN PLAIN WORDS**
>
> The software comes in two layers:
>
> - **the pipeline itself**, a small program that organises everything (the `rnaseq-pipeline` command);
> - **the scientific tools** it drives (FastQC, HISAT2, DESeq2, …), installed with a package manager called **conda / mamba**, into two ready-made environments named `rnaseq-tools` and `rnaseq-r`.

## Step 1 · Install Miniforge (the package manager)

Miniforge installs conda and mamba, which download and install scientific software without administrator rights. Skip this step if you already have conda or mamba.

```
cd ~
curl -L -O https://github.com/conda-
forge/miniforge/releases/latest/download/Miniforge3-Linux-x86_64.sh
bash Miniforge3-Linux-x86_64.sh -b -p ~/miniforge3
```

`cd ~` goes to your home folder; `curl` downloads the installer; `bash …` runs it and installs into the folder `miniforge3` in your home folder.

### Step 2 · Download the pipeline from GitHub

The pipeline's source code is kept on GitHub. `git clone` downloads a copy of it into a folder called `rnaseq_pipeline`:

```
cd ~
git clone https://github.com/vipin839/rnaseq_pipeline.git
```

If your GitHub account has an SSH key, the SSH address works too: `git clone git@github.com:vipin839/rnaseq_pipeline.git`. To get exactly version 1.1.1, add `--branch v1.1.1`. If the computer says git is not installed: `sudo apt install git`.

### Step 3 · Install the rnaseq-pipeline command

**pipx** installs a Python program in its own private space and makes its command available everywhere. On Ubuntu, install pipx once (this one step asks for your password):

```
sudo apt install -y pipx
pipx ensurepath
```

Close the Ubuntu window and open it again, so the new command location is known.

Then install the pipeline from the folder you downloaded:

```
pipx install ~/rnaseq_pipeline
```

Check it worked: `rnaseq-pipeline --version` should print `Pipeline Version: 1.1.1`.

> **TIP**
> Without pipx, you can also run the pipeline straight from the downloaded folder: `~/rnaseq_pipeline/rnaseq_pipeline`. It then needs one small library: `sudo apt install python3-yaml`.

### Step 4 · Install the scientific tools

The pipeline carries two recipe files listing exactly the tested tool versions. These commands read them and build the two environments. They download a few gigabytes and take 10–30 minutes:

```
~/miniforge3/bin/mamba env create -f "$(rnaseq-pipeline --runtime-env tools)"
~/miniforge3/bin/mamba env create -f "$(rnaseq-pipeline --runtime-env r)"
```

The first creates `rnaseq-tools` (FastQC, MultiQC, fastp, HISAT2, samtools, StringTie, featureCounts, SRA Toolkit, RSeQC); the second creates `rnaseq-r` (R with DESeq2 and the plotting packages). The pipeline finds them automatically.

> **TIP**
>
> You can also do this step from inside the program: **Main menu → 4. Manage Dependencies → Install missing components**. It shows every command and asks before running it. On older processors (before about 2013) it automatically chooses a compatible StringTie version.

## Step 5 · Your personal settings (NCBI e-mail)

NCBI asks everyone who searches or downloads from it to give an e-mail address. You can also add a free **API key** (from your NCBI account page), which allows faster searches. These settings belong to you, not to a project, and are never copied into projects, reports or logs. Create the settings file with the simple text editor nano:

```
mkdir -p ~/.config/rnaseq-pipeline
nano ~/.config/rnaseq-pipeline/config.yaml
```

Type these lines into the editor (with your own e-mail; leave the key empty if you have none), then save with **Ctrl + O**, **Enter**, and leave with **Ctrl + X**:

```
ncbi:
  email: "you@example.org"
  api_key: ""
projects_dir: ~/rnaseq_projects
reference_store: ~/rnaseq_references
```

Keep the two spaces before `email` and `api_key`. `projects_dir` is where projects are created; `reference_store` is where genomes are kept and shared between projects.

## Step 6 · Health check

The health check tests your computer, the settings and every tool, and runs a tiny real analysis through all of them, including DESeq2:

```
rnaseq-pipeline --check
```

| Result | Meaning | What to do |
|---|---|---|
| **HEALTH: PASS** | everything works | start your first project |
| **HEALTH: WARNING** | it works, with a remark (for example, NCBI e-mail not set) | read the remark; you can usually continue |
| **HEALTH: FAIL** | a tool is missing or broken | the failed line names the component and the fix; often Main menu → 4 repairs it |

# 6. Starting the program

Type the command and press Enter. The program shows a menu; you answer by typing the number of your choice and pressing Enter.

```
rnaseq-pipeline
```

| Main menu | What it is for |
|---|---|
| **1. Start New RNA-seq Project** | begin a new analysis (Chapters 7–12 follow this path) |
| **2. Resume Existing Project** | continue, re-run or change a project you started before |
| **3. Validate Existing Project** | re-check a project from top to bottom and get a health verdict |
| **4. Manage Dependencies / Environment** | install or repair the scientific tools; export version records |
| **5. Manage Reference Databases** | download and prepare genomes ahead of time; see or delete prepared ones |
| **6. View Previous Reports** | open the HTML report of an earlier project |
| **7. Pipeline Configuration** | view the default settings or edit a project's settings |
| **8. Exit** | leave the program |

## Messages on the screen

| Tag | Meaning |
|---|---|
| [INFO] | information |
| [RUNNING] | a step is working |
| [OK] | a result was checked and is correct |
| [WARNING] | worth your attention, but not a stop |
| [ERROR] / [FAILED] | the step stopped; the screen says why and what to do |
| [SKIPPED] | not needed (already done and still valid, or not requested) |
| [ACTION] | a recommendation for you |

## Every step has the same screen

Before each step the pipeline shows what it will do, how many samples are ready, which reference is used and how much disk space is needed, and offers: **start**, **review settings**, **change settings**, **validate inputs again**, **return to main menu** or **cancel**. Nothing heavy starts without you.

```
STEP 7 — ALIGNMENT (HISAT2)
Samples: 6   Validated: 6/6   Reference: GRCh38 / GENCODE 46   Threads: 11   Estimated
storage: 40.3 GB
1. Start alignment  2. Review settings  3. Change settings  4. Validate inputs again
5. Return to main menu  6. Cancel
```

An example of a step screen.

## Useful start options

| Type | What it does |
|---|---|
| `rnaseq-pipeline --project ~/rnaseq_projects/RNAseq_MyStudy` | open a project directly |
| `rnaseq-pipeline --auto` | skip the screens of quick steps; decisions and long steps still ask |
| `rnaseq-pipeline --dry-run --project DIR` | show what would run, without running anything |
| `rnaseq-pipeline --validate-project DIR` | re-check a project without menus |
| `rnaseq-pipeline --check` | health check of the installation |
| `rnaseq-pipeline --verbose` | also print every command the tools receive |

# Part III · The workflow, step by step

## 7. The workflow at a glance

The complete analysis has 17 steps in five phases. Three of them (yellow diamonds) are decisions you make; everything else runs on its own, step by step, each one checked before the next begins.

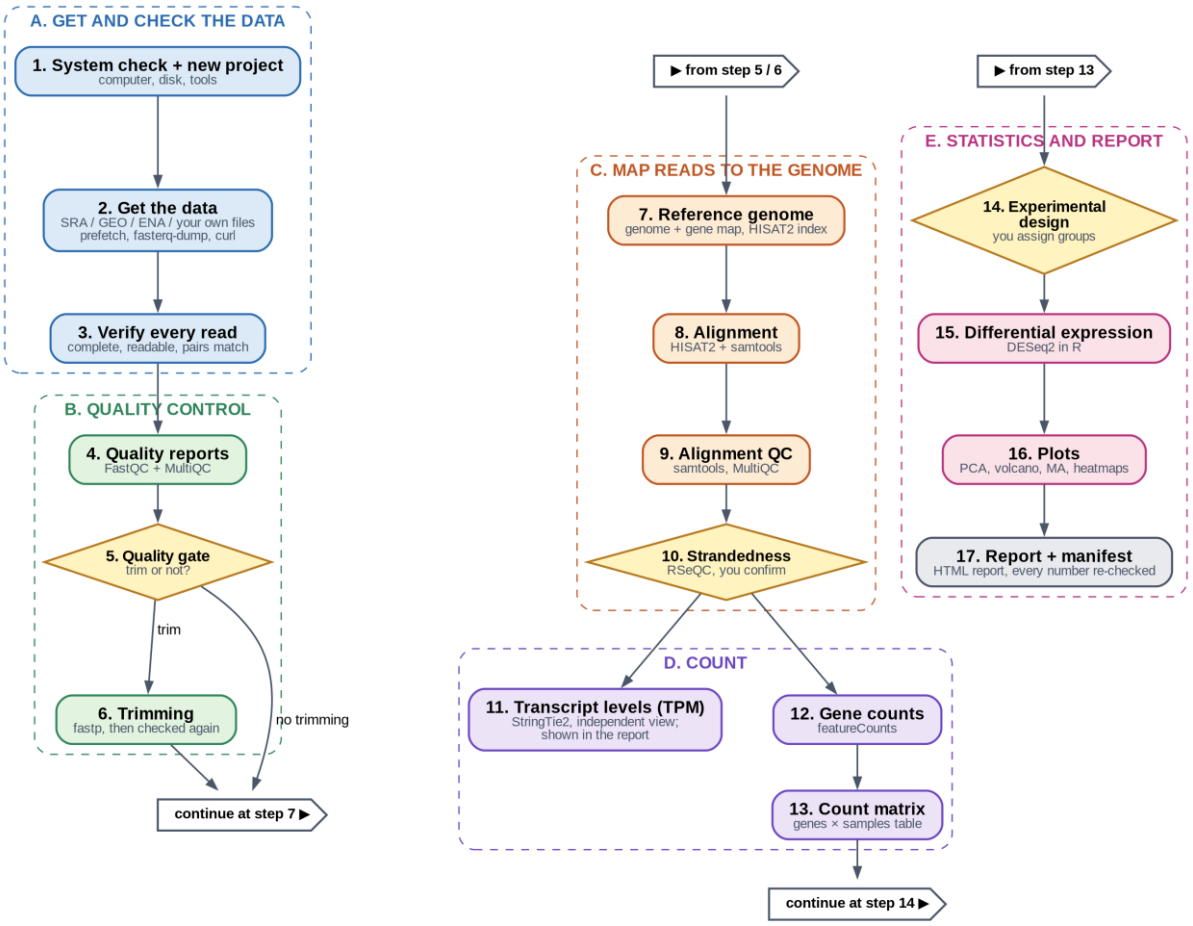



**Figure 4.** The complete workflow. Colours mark the phases; yellow diamonds are your decisions. Step 11 (StringTie2) gives an independent view that appears in the report; the statistics use step 12.

### A new project, from start to finish

1. **Main menu → 1. Start New RNA-seq Project.** The computer is checked (step 1).
2. **Name the project.** Letters, digits, dot, underscore and hyphen only. A folder `RNAseq_<name>` is created in your projects folder.
3. **Choose the data:** search NCBI, give an accession (SRA, GEO, ENA) or point to your own FASTQ files. Review the table of samples; choose **full data** or a **pilot subset** for a quick test.
4. **Choose the organism and reference** (from the catalogue, any organism on NCBI, or your own files).
5. **Optional databases** (GO/Reactome enrichment) — off by default.
6. **Choose the samples** to use (all, or a subset).

7. The pipeline now walks through the steps. It stops for the **quality gate** (step 5), **strandedness** (step 10) and the **experimental design** (step 14).
8. At the end, open `reports/final_pipeline_report.html` in your web browser.

> **TIP**
> You can leave at any time (Return to main menu, or close the window). Next time choose **Main menu → 2. Resume Existing Project**; finished steps are re-checked and skipped.

# 8. Steps 1–3 · Getting and checking the data

## Step 1 · System check and new project

| | |
|---|---|
| **Tools** | the pipeline itself |
| **Checks** | operating system, processor, memory, free disk, file system, Python, R, conda |
| **Output** | the project folder with its sub-folders and a project record |
| **Your role** | give the project a name |

Before anything is downloaded, the pipeline looks at the computer. Problems that would ruin an analysis hours later — too little disk, a read-only folder, a FAT32 USB disk — stop it here, with an explanation.

## Step 2 · Getting the data

| | |
|---|---|
| **Tools** | NCBI E-utilities (search), SRA Toolkit: prefetch, vdb-validate, fasterq-dump; curl (ENA); pigz (compression) |
| **Input** | keywords, an accession (SRR/SRX/SRP/PRJNA/GSE/ERR…) or your own FASTQ files |
| **Output** | one FASTQ file per sample (two for paired-end) in `data/`, plus a metadata table |
| **Your role** | choose the source, review the sample table, choose full data or a pilot subset |

There are five ways to bring data in:

| Source | What you give | How it is fetched |
|---|---|---|
| **Search NCBI** | keywords and filters (organism, platform, date…) | results are listed page by page; pick a study and its runs are loaded |
| **NCBI SRA** | run, experiment, study or BioProject accession | downloaded with prefetch, checked with vdb-validate, converted to FASTQ with fasterq-dump |
| **NCBI GEO** | a series number (GSE…) | samples are linked to their SRA runs; GEO sample descriptions are shown to help you |
| **ENA** | any ENA or SRA accession | FASTQ files downloaded directly; each file's MD5 checksum is verified |
| **Your own files** | files or a folder of FASTQ files | linked (not copied) into the project; the originals are never changed. Pairs are recognised from names like `_R1/_R2` or `_1/_2` |

> **IN PLAIN WORDS**
>
> An **accession** is the public ID of a dataset, like a library catalogue number. SRR… is one sequencing run, GSE… a whole GEO study, PRJNA… a BioProject.

YOUR DECISION

**Full data or pilot subset?** A pilot keeps only the first N reads of each sample. It is fast and good for testing the set-up, but **not** for conclusions: the report and manifest carry a clear PILOT warning.

**One organism and one read layout per project.** A selection mixing organisms, or single- and paired-end runs, is refused.

WHAT THE PIPELINE CHECKS

Downloads are resumable and restart cleanly; files are only accepted after their checksum or size is verified.

Single-cell datasets are recognised (from metadata and read structure) and refused.

Every input file gets a fingerprint in the manifest.

## Step 3 · Verifying every read

| | |
|---|---|
| **Tools** | the pipeline's own FASTQ reader (streaming, constant memory) |
| **Input** | the FASTQ files |
| **Output** | `data/metadata/fastq_validation_report.tsv`: reads and status per file |
| **Your role** | none, unless a sample fails (then: exclude it or stop) |

Every single record of every file is read. A FASTQ record has four lines: a name starting with @, the letters, a + line, and one quality character per letter. The pipeline checks that each record is complete, that only valid letters and quality characters appear, that quality and sequence have the same length, and that compressed files are not damaged. For paired-end data, both files must contain the same number of reads and the mates must belong together.

IN PLAIN WORDS

This is like checking that a delivery is complete and undamaged before you start cooking. A truncated download would otherwise give silently wrong counts weeks later.

# 9. Steps 4–6 · Quality control and trimming

## Step 4 · Quality reports

| | |
|---|---|
| **Tools** | **FastQC** (one report per FASTQ file), **MultiQC** (one combined report for all samples) |
| **Input** | the verified FASTQ files |
| **Output** | `qc/fastqc_raw/` and `qc/multiqc_raw/multiqc_report.html` |
| **Your role** | none (you may open the MultiQC report to look) |

FastQC draws a health profile of each file: the quality of each position along the reads, how much adapter sequence is present, the GC content, duplicated and over-represented sequences, and unknown letters (N). MultiQC puts the profiles of all samples side by side, so an odd sample stands out.

> **WHAT THE PIPELINE CHECKS**
> FastQC's read total must equal the number of reads counted in step 3, and each report must name the file it analysed — so a report can never be mixed up with another sample's.

## Step 5 · The quality gate

| | |
|---|---|
| **Tools** | the pipeline (reads FastQC's numbers and compares them with limits) |
| **Output** | a status per sample and your decision, saved in `qc/assessment/qc_decision.json` |
| **Your role** | **decide whether to trim** |

FastQC's own red crosses are not a good guide for RNA-seq: some of them are normal (for example high duplication, because very active genes produce many identical reads). The quality gate therefore uses limits chosen for RNA-seq and separates **technical problems** (which trimming can fix) from **patterns that are normal** for RNA-seq.

**Table 3.** Quality-gate limits (all can be changed in the settings).

| What is measured | REVIEW if | TRIMMING RECOMMENDED if | Why |
|---|---|---|---|
| Adapter content | > 1 % | > 5 % | adapter letters are not from your RNA and hinder alignment |
| Quality at the read ends (last 10 %) | below Q28 | below Q20 | Q20 = 1 error in 100 letters; poor ends are trimmed |
| Lowest quality at any position | below Q10 | – | a failed sequencing cycle |
| Over-represented sequence | > 1 % | – | often a very active gene, so only a review |
| Unknown letters (N) | > 5 % | – | the machine could not read a position |
| Duplication | > 80 % | – | normal for highly expressed genes; not a trimming issue |
| GC differs from the other samples | by > 10 points | – | possible contamination or rRNA |
| Number of reads | < 1 million | – | too few reads give little statistical power |

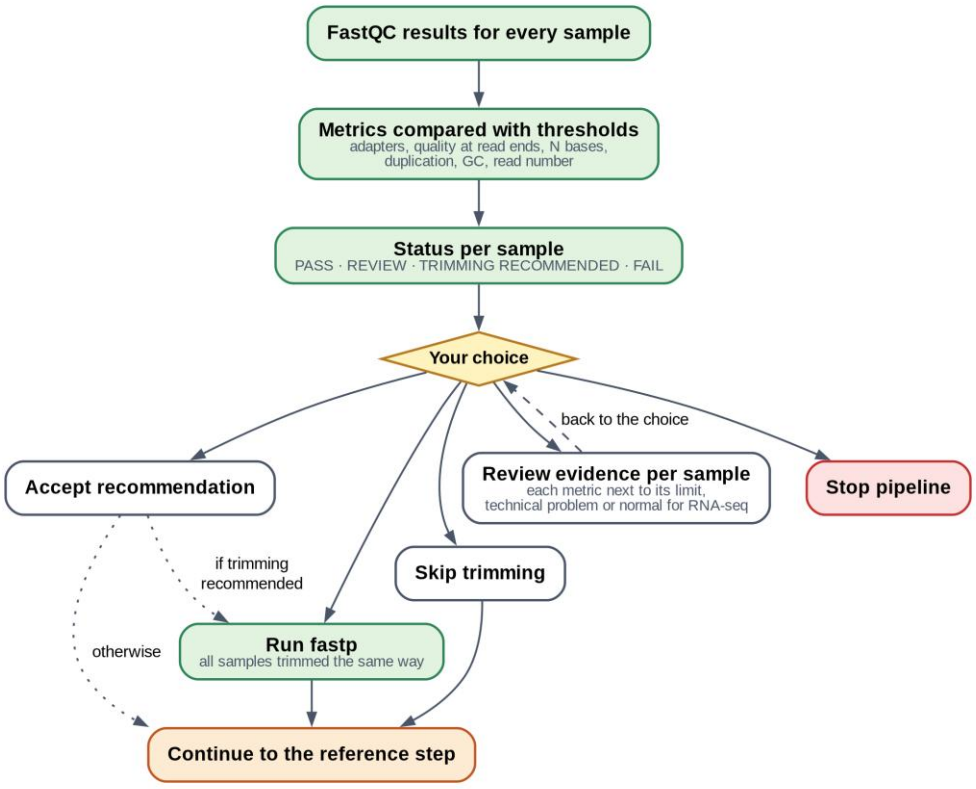



**Figure 5.** The quality gate.

YOUR DECISION

**Accept recommendation** — trim if any sample needs it (then all samples are trimmed the same way, so they stay comparable).

**Run fastp** / **Skip trimming** — your own choice.

**Review the evidence per sample** — every measurement next to its limit, with an explanation; then back to the same choice.

**Stop pipeline** — for example to replace a bad sample.

## Step 6 · Trimming (only if chosen)

| | |
|---|---|
| **Tool** | **fastp** |
| **Input** | the raw FASTQ files |
| **Output** | trimmed FASTQ files in `data/trimmed/`, fastp reports, and new FastQC/MultiQC reports |
| **Your role** | none |

fastp removes adapter sequence and cuts away poor-quality ends: it slides a window of 4 letters from the end of each read and cuts while the average quality is below 20. Reads that become shorter than 25 letters, or that are mostly low-quality, are removed. Typically 90–98 % of reads are kept.

WHAT THE PIPELINE CHECKS

The trimmed files are verified record by record like in step 3; their read count must equal what fastp reports; and quality control runs again on them.

# 10. Steps 7–10 · Mapping reads to the genome

## Step 7 · Reference genome

| | |
|---|---|
| **Tools** | samtools faidx, HISAT2 (hisat2-build, splice-site and exon extraction), the pipeline's own checks |
| **Input** | a genome (FASTA) and a gene annotation (GTF) — downloaded or your own |
| **Output** | a prepared reference in the shared store `~/rnaseq_references`, linked into the project |
| **Your role** | choose the organism and source when creating the project; confirm the download |

To place reads you need the organism's **genome** (the full DNA sequence, a FASTA file) and its **gene annotation** (a GTF file: a map saying where each gene and exon lies). Sources: GENCODE, Ensembl, RefSeq and UCSC for human, mouse and yeast; **any organism** through an NCBI search; or your own matching files.

HISAT2 then builds an **index**: a compressed, searchable form of the genome, like the index of a book, that lets millions of reads be placed quickly. The index is built once and reused by every project.

> **WHAT THE PIPELINE CHECKS**
>
> Downloads are checked against the provider's checksums.
>
> Genome and annotation must belong together: same chromosome names, genes inside chromosome lengths, same assembly. A human GRCh37 annotation on a GRCh38 genome, for example, is caught here.
>
> The finished index is inspected and must contain exactly the genome's sequences.

> **IN PLAIN WORDS**
>
> Bacteria: their annotations often describe genes as coding sequences (CDS) only. The pipeline notices this, asks you, counts CDS instead, and adjusts the later steps.

## Step 8 · Alignment

| | |
|---|---|
| **Tools** | **HISAT2** (aligner) piped into **samtools** (sorting and indexing) |
| **Input** | the (trimmed) FASTQ files and the HISAT2 index |
| **Output** | one sorted, indexed **BAM** file per sample in `alignment/`, and an alignment summary |
| **Your role** | none |

HISAT2 takes every read and finds the place in the genome where it matches best. In plants and animals genes are interrupted by introns, which are removed from mRNA; a read can therefore span two exons. HISAT2 is a **splice-aware** aligner: it can split such a read across the intron (the reads-to-counts picture in Chapter 2 shows one). The result is a **BAM** file: every read with its position, sorted along the genome.

**WHAT THE PIPELINE CHECKS**

Every program in the chain is checked, not only the last one.

The number of reads in the BAM must equal the number of reads that went in — a truncated file is caught and set aside as `.FAILED.bam`.

Alignment rate below 20 % stops the sample (wrong organism?); below 70 % gives a warning.

## Step 9 · Alignment quality

| | |
|---|---|
| **Tools** | **samtools** idxstats, flagstat and stats; **MultiQC** |
| **Output** | `bam_qc_summary.tsv` and a MultiQC report of the alignments |
| **Your role** | none |

This step summarises how the reads were placed: how many aligned once, several times or not at all, how they are spread over chromosomes, and for paired-end data whether mates aligned together.

## Step 10 · Strandedness

| | |
|---|---|
| **Tool** | **RSeQC** (infer_experiment) |
| **Input** | the BAM files and the gene map |
| **Output** | forward, reverse or unstranded, with the evidence per sample |
| **Your role** | **confirm the result, or choose** |

RSeQC looks at 200,000 aligned reads per sample and asks: do they lie on the same strand as the gene they overlap, or on the opposite strand? With a stranded kit, almost all reads point one way.

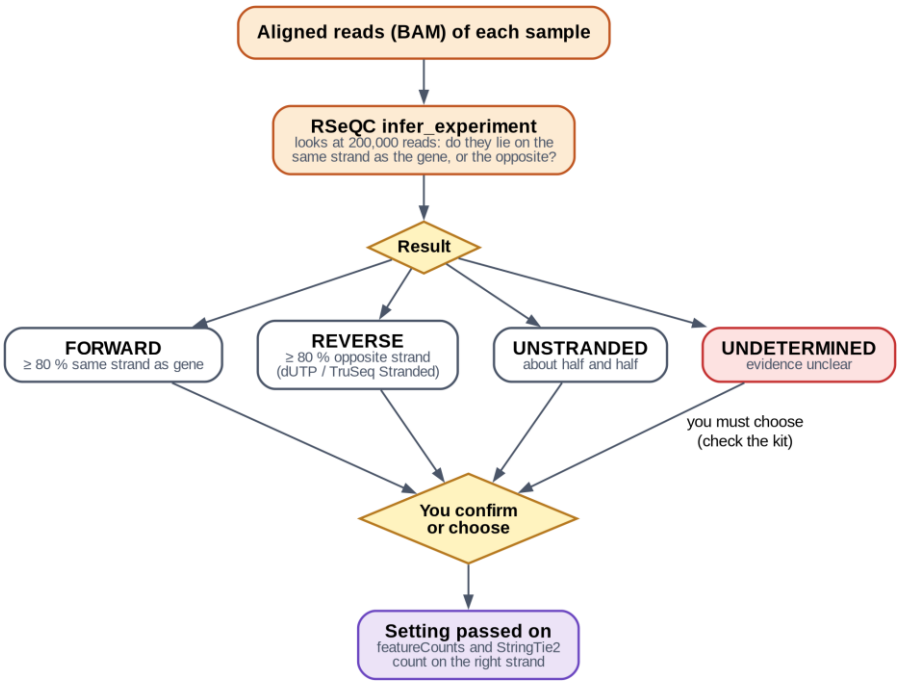



**Figure 6.** How strandedness is decided.

| Call | Rule | Typical kits | Used as |
|---|---|---|---|
| Forward | ≥ 80 % of reads on the gene's strand | some ligation-based kits | featureCounts -s 1, StringTie --fr |
| Reverse | ≥ 80 % on the opposite strand | dUTP methods, Illumina TruSeq Stranded | featureCounts -s 2, StringTie --rf |
| Unstranded | the two shares differ by ≤ 10 points | older or non-stranded kits | featureCounts -s 0 |
| Undetermined | more than half of reads uninformative, or samples disagree | – | you must choose |

**YOUR DECISION**

Confirm the inferred value, or pick one yourself. If unsure, look up the library kit in the study's methods. A wrong choice can halve your counts, so it is never guessed silently.

# 11. Steps 11–13 · Counting

## Step 11 · Transcript abundance (StringTie2)

| | |
|---|---|
| **Tool** | **StringTie2** |
| **Input** | BAM files and the gene annotation |
| **Output** | transcript and gene abundance tables in **TPM** (`stringtie/merged/`) |
| **Your role** | none |

StringTie2 estimates how abundant each known transcript and gene is, as **TPM** (transcripts per million). TPM is useful for comparing the level of different genes within a sample ("is gene X more abundant than gene Y?"). Here it runs only on known transcripts, so all samples are directly comparable.

> **IMPORTANT**
> TPM values are **not** used for the statistical test. DESeq2 needs raw counts (step 12). Use TPM for description and visualisation, not for testing differences.

## Step 12 · Gene counts (featureCounts)

| | |
|---|---|
| **Tool** | **featureCounts** (from the Subread package) |
| **Input** | BAM files, the gene annotation, the strandedness |
| **Output** | `featurecounts/featurecounts.txt` (reads per gene) and a summary per sample |
| **Your role** | none |

featureCounts goes through every aligned read and adds it to the gene whose exons it overlaps, on the correct strand. For paired-end data, a pair counts once (it is one RNA fragment). Reads that align to several places, overlap two genes, or fall outside genes are **not** counted, and the summary says how many of each there were.

> **WHAT THE PIPELINE CHECKS**
> The number of genes must equal the number of genes in the annotation.
>
> featureCounts' totals are reconciled with the reads that entered alignment; a loss or duplication of reads between steps stops the analysis.
>
> Less than 30 % of reads assigned to genes gives a warning (wrong strandedness? rRNA? incomplete annotation?).

## Step 13 · Count matrix

| | |
|---|---|
| **Tool** | the pipeline |
| **Output** | `counts/gene_count_matrix.tsv` (and .csv): one row per gene, one column per sample |
| **Your role** | if several runs belong to one biological sample (lanes), choose whether to add them up |

The count matrix is the central table of the whole analysis: whole numbers of reads per gene per sample. It is what DESeq2 analyses, and you can open it in Excel.

**Table 4.** A count matrix (made-up numbers). Gene A looks about 3× higher in the treated samples; gene B looks unchanged; gene C has too few reads to test.

| gene_id | Control_1 | Control_2 | Control_3 | Treated_1 | Treated_2 | Treated_3 |
|---|---|---|---|---|---|---|
| Gene A | 512 | 468 | 530 | 1490 | 1622 | 1388 |
| Gene B | 88 | 95 | 71 | 90 | 84 | 102 |
| Gene C | 0 | 2 | 1 | 0 | 0 | 1 |

**WHAT THE PIPELINE CHECKS**

Every column total must equal the number of reads featureCounts assigned for that sample. Counts must be whole, non-negative numbers; no sample may be empty.

# 12. Steps 14–17 · Statistics, plots and report

## Step 14 · Experimental design

| | |
|---|---|
| **Tool** | the pipeline (checks), R (checks again) |
| **Output** | `counts/sample_metadata.tsv` and `counts/design.json` |
| **Your role** | **tell the pipeline which sample is in which group, and what to compare** |

The statistics need to know the groups. The pipeline never assumes them from sample names; you assign them, in whichever way is easiest:

- **one by one** — type the group of each sample;
- **by pattern** — e.g. every sample whose name or description contains "WT" is in group WT;
- **from a public metadata field** — choose a field from the GEO/SRA description (shown with its values);
- **from a file** — a table (TSV/CSV) with the columns sample and condition (and optionally batch, sex…).

Then you set:

| Setting | Meaning | Example |
|---|---|---|
| **Condition** | the groups you want to compare | WT, mutant |
| **Covariates** (optional) | other known differences to account for | batch 1/2, sex |
| **Formula** | what explains the counts | `~ condition`, or `~ batch + condition` |
| **Reference (baseline) level** | the group everything is compared against | WT (control) |
| **Contrasts** | which comparisons to test | mutant vs WT |

> **IN PLAIN WORDS**
>
> With `~ batch + condition`, DESeq2 first accounts for differences between batches and then tests the condition. This only works if each batch contains both conditions.

> **WHAT THE PIPELINE CHECKS**
>
> Every group needs at least 2 replicates.
>
> Batch and condition must not be confounded (if all controls are batch 1 and all treated batch 2, the two effects cannot be separated, and the design is refused with this explanation).
>
> The design must be **confirmed**; if the sample sheet is edited afterwards, it must be confirmed again.

## Step 15 · Differential expression (DESeq2)

| | |
|---|---|
| **Tools** | **R** and **DESeq2**; **apeglm** for fold-change shrinkage |
| **Input** | the count matrix and the confirmed design |
| **Output** | `results/deseq2/<comparison>/` — full results, significant, up and down genes, statistics |
| **Your role** | none (thresholds can be changed in the settings) |

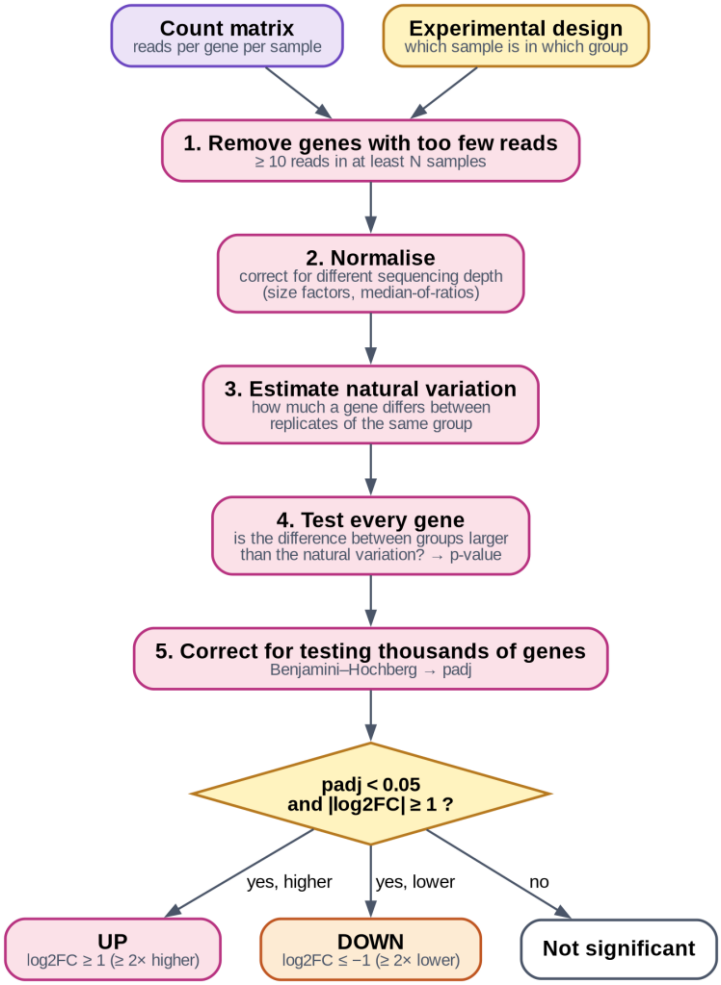



**Figure 7.** What DESeq2 does with your counts.

In five steps:

1. **Filter.** Genes with fewer than 10 reads in at least as many samples as the smallest group are removed: they carry too little information to test.
2. **Normalise.** Samples are sequenced to different depths (one may have 20 million reads, another 30 million). DESeq2 calculates a **size factor** per sample so that counts become comparable.
3. **Estimate natural variation** (dispersion) of each gene from the replicates, borrowing information across genes so that even 3 replicates give stable estimates.
4. **Test every gene:** is the difference between the groups larger than the natural variation would explain? The answer is a **p-value**.
5. **Correct for multiple testing.** With 20,000 genes tested, about 1,000 would reach $p < 0.05$ by pure chance. The Benjamini–Hochberg method turns p-values into **adjusted p-values (padj)** that control the **false discovery rate**: among genes with padj < 0.05, about 5 % are expected to be false positives.

A gene is called **up-regulated** if padj < 0.05 **and** log2 fold change ≥ 1 (at least twice as high), and **down-regulated** if padj < 0.05 **and** log2 fold change ≤ −1 (at most half). Both thresholds can be changed (Main menu → 7).

**WHAT THE PIPELINE CHECKS**

After R finishes, the pipeline recalculates in Python — without trusting R — which genes should have been tested, the size factors, the normalised counts and the direction of every comparison. A swapped baseline (all fold changes reversed) or a missing gene stops the step.

## Step 16 · Plots

| | |
|---|---|
| **Tools** | R: ggplot2, pheatmap, EnhancedVolcano |
| **Output** | PNG (300 dpi) and PDF in `results/plots/`; the data behind every plot in `results/tables/` |
| **Your role** | none |

Overview plots for all samples (PCA, sample distance and correlation heatmaps, library sizes, count distributions) and, for every comparison, an MA plot, a volcano plot and two heatmaps. Chapter 15 explains how to read each one. Each plot's data table is checked against the result tables, so a plot can never show different genes than the lists.

## Step 17 · Report and manifest

| | |
|---|---|
| **Tool** | the pipeline |
| **Output** | `reports/final_pipeline_report.html` and the manifest in `pipeline_manifest/` |
| **Your role** | open the report in a web browser |

The report has one section per step: project overview, input data, design, reference, software versions, quality control, trimming, alignment, strandedness, StringTie2, featureCounts, count matrix, DESeq2 results with plots, warnings, parameters and reproducibility information. The manifest records the software versions, settings, design, input-file fingerprints and genome checksums.

## The chain of checks

Each number is followed from one step to the next. If any link breaks — a file cut short by a full disk, a process killed half-way — the analysis stops at that point instead of producing wrong results.

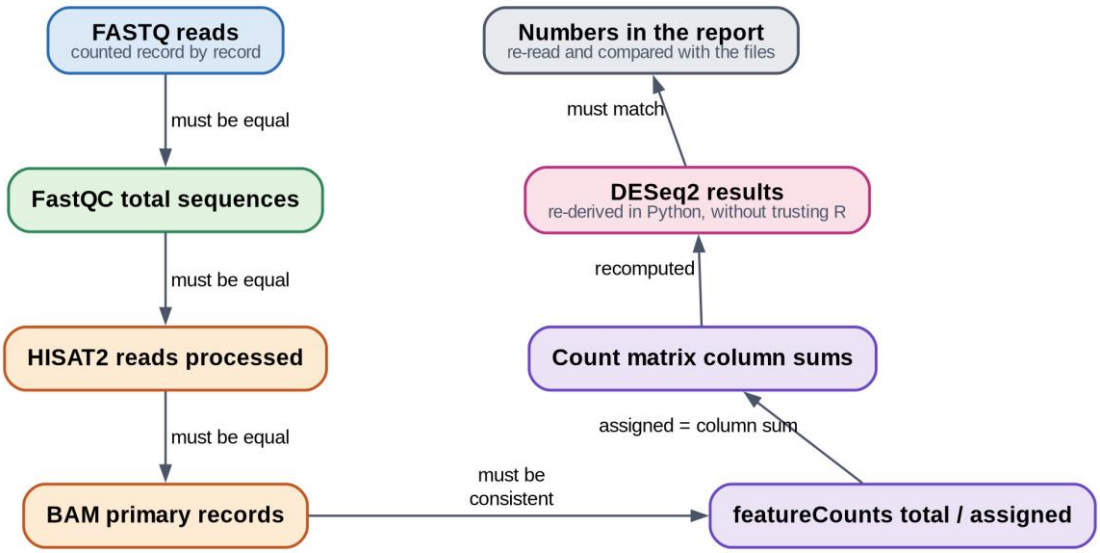



**Figure 8.** The chain of checks that connects the steps.

# 13. The decisions you make

Most of the analysis is automatic. These are the moments where the pipeline waits for you, because the right answer depends on your experiment.

**Table 5.** Decisions during an analysis.

| When | Question | How to decide |
|---|---|---|
| Data (step 2) | Full data or a pilot subset? | Pilot to test the set-up quickly; full data for any result you will use. |
| Data (step 2) | Which samples? | All, or a subset for testing. Failed samples can be excluded and re-included later. |
| Reference | Which genome and annotation? | The organism of your samples; for public data check the metadata table. Use one provider for both genome and annotation. |
| Quality gate (step 5) | Trim or not? | Accept the recommendation unless you have a reason. Use Review to see why. |
| Lanes (step 13) | Add up runs of the same sample? | Yes for technical replicates / lanes of one library. |
| Strandedness (step 10) | Forward, reverse or unstranded? | Confirm the inferred value; if undetermined, check the kit. |
| Design (step 14) | Groups, baseline, comparisons? | From your experiment. The baseline is usually the control. |
| Before DESeq2 | Proceed? | A summary of everything so far is shown; confirm to run the statistics. |

# Part IV · Results

## 14. Where your results are

Every project is a folder (by default in `~/rnaseq_projects`). On Windows with WSL you can open it in File Explorer: type `\\wsl$` in the address bar, then Ubuntu → home → your user name → rnaseq_projects.

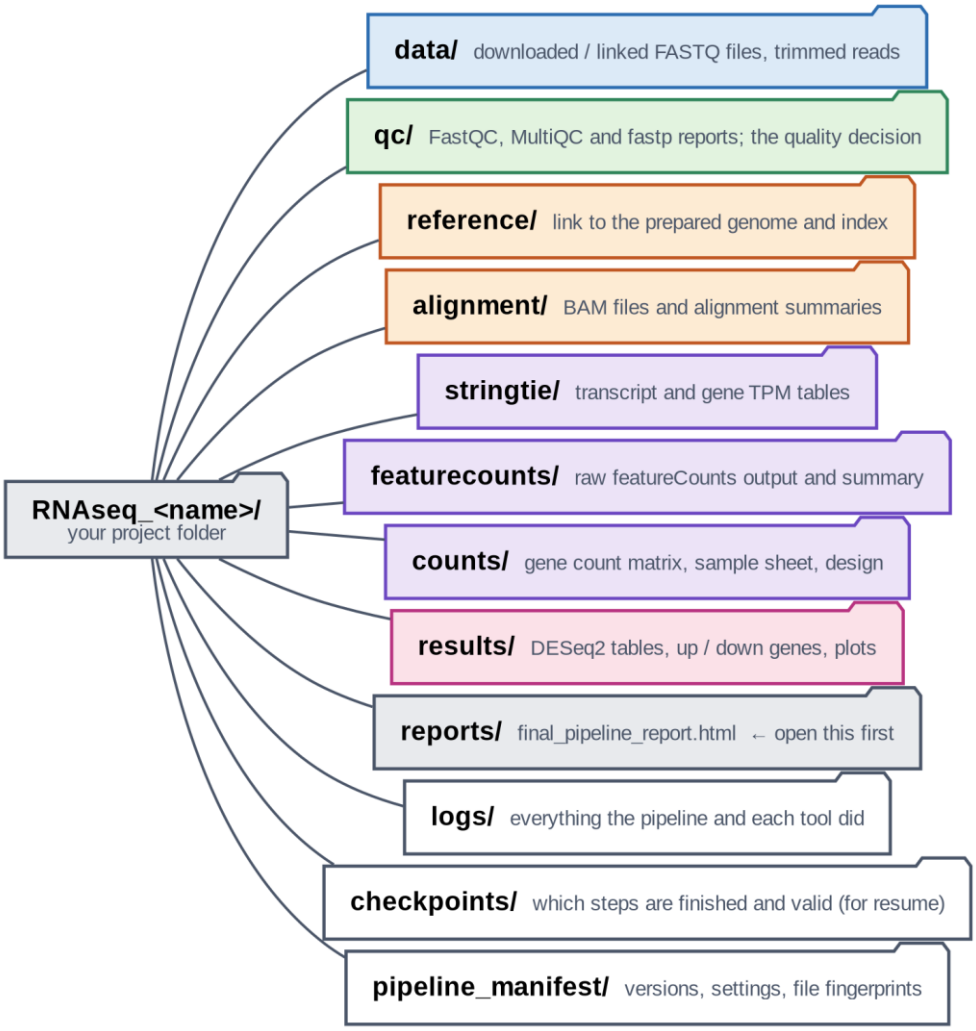



**Figure 9.** The project folder.

| What you want | Where it is |
|---|---|
| **The report (start here)** | `reports/final_pipeline_report.html` |
| Up- and down-regulated genes | `results/deseq2/<comparison>/upregulated.tsv`, `downregulated.tsv` |
| All tested genes with statistics | `results/deseq2/<comparison>/full_results.tsv` |
| Significant genes only | `results/deseq2/<comparison>/significant_results.tsv` |
| Short summary of a comparison | `results/deseq2/<comparison>/statistics.txt` |
| Plots (PNG and PDF) | `results/plots/` and `results/plots/<comparison>/` |
| Normalised counts | `results/deseq2/normalized_counts.tsv` |
| Raw count matrix | `counts/gene_count_matrix.tsv` (also .csv) |
| TPM per gene / transcript | `stringtie/merged/` |
| Quality reports | `qc/multiqc_raw/`, `qc/multiqc_trimmed/`, `qc/multiqc_alignment/` |
| Everything that happened | `logs/pipeline.log`; problems only: `logs/pipeline_errors.log` |

> **TIP**
>
> The .tsv files are tables: open them in Excel or LibreOffice (choose "tab" as separator if asked).

# 15. Reading your results

## First, check the quality numbers

Before you look at gene lists, make sure the data behaved well. The report shows these numbers per sample. The ranges below are general guidance; what is normal also depends on the organism and the kit.

**Table 6.** Quality numbers and what they mean.

| Number (report section) | Good | Look closer | Possible reasons for low values |
|---|---|---|---|
| Reads per sample (Input data) | ≥ 10–20 million (human, mouse); fewer suffice for small genomes | < 5 million; very uneven between samples | shallow sequencing; failed library |
| Reads kept after trimming (Trimming) | > 90 % | < 80 % | very short inserts, poor run |
| Overall alignment rate (Alignment) | > 80 % | < 70 % (warning); < 20 % stops | wrong organism or reference, contamination |
| Assigned to genes (featureCounts) | > 60 % | < 30 % (warning) | wrong strandedness, rRNA, incomplete annotation |
| Strandedness fractions | one side ≥ 80 %, or both ~50 % | in between | degraded RNA, mixed kits |

## The result table, column by column

**Table 7.** Columns of full_results.tsv.

| Column | What it means | How to use it |
|---|---|---|
| gene_id | the gene's identifier from the annotation | look it up in Ensembl, NCBI or TAIR for the name and function |
| baseMean | average normalised count over all samples | how strongly the gene is expressed overall |
| log2FoldChange | change between groups, on a log2 scale (see Table below) | size and direction of the change |
| lfcSE | uncertainty of the log2 fold change | smaller = more precise |
| stat | the test statistic (fold change / its uncertainty) | rarely needed |
| pvalue | chance of seeing such a difference if nothing changed | do **not** use alone for thousands of genes |
| padj | p-value corrected for testing many genes | **the value to use**: padj < 0.05 = significant |
| log2FoldChange_shrunk | fold change pulled towards 0 when evidence is weak | good for ranking and plots |
| regulation | up, down or ns (not significant) | the pipeline's call with your thresholds |

**Table 8.** Translating log2 fold changes. Positive = higher in the test group than in the baseline.

| log2 fold change | means | log2 fold change | means |
|---|---|---|---|
| 1 | 2× higher | −1 | 2× lower (half) |
| 2 | 4× higher | −2 | 4× lower |
| 3 | 8× higher | −3 | 8× lower |
| 0.58 | 1.5× higher | 0 | no change |

> **IN PLAIN WORDS**
>
> Why log2? Doubling and halving are equally big changes in biology. On the log2 scale they become +1 and −1, which is symmetric and easy to read.

## The plots

The examples below come from a real analysis made with this pipeline (described in Chapter 16): Arabidopsis wild type (WT) versus the *feronia* (fer) mutant, three replicates each.

## PCA plot — do the samples group as expected?

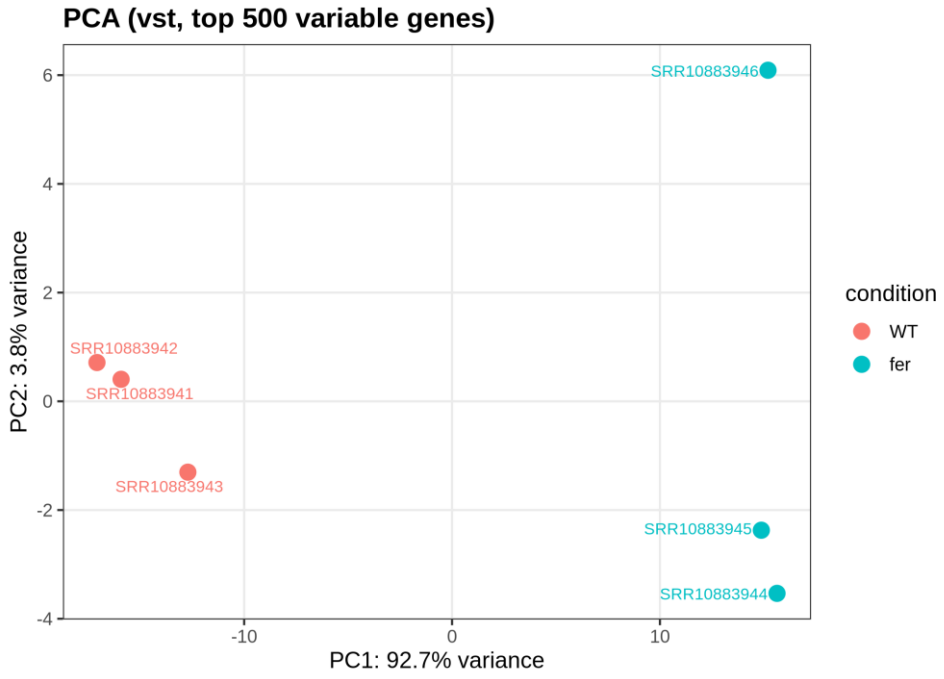



**Figure 10.** PCA of six Arabidopsis samples. WT (red) and fer (blue) separate clearly along PC1.

Each point is a sample. Samples with similar overall expression lie close together. PC1 (horizontal) is the direction of the largest differences between samples; the percentage says how much of the variation it explains.

- **Good sign:** replicates of a group cluster together and the groups separate — as here, where 92.7 % of the variation lies between WT and fer.

- **Warning sign:** a sample sitting with the wrong group (possible sample swap), or groups not separating at all (little effect), or separation by batch rather than by condition.

## Sample distance heatmap — which samples are most alike?

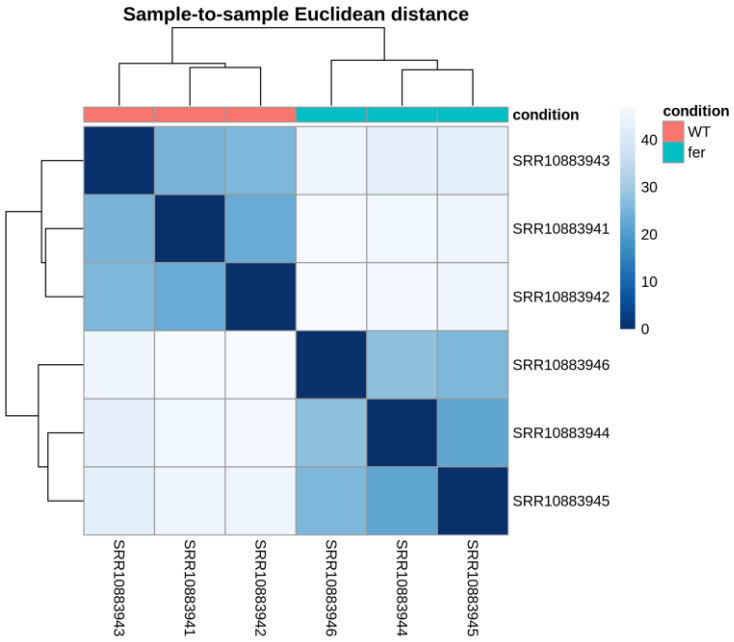



**Figure 11.** Sample-to-sample distances. Darker = more similar.

The same question as PCA, as a grid: each square compares two samples. Replicates of a group should form dark blocks; the tree above shows which samples cluster.

## Volcano plot — which genes changed, and how strongly?

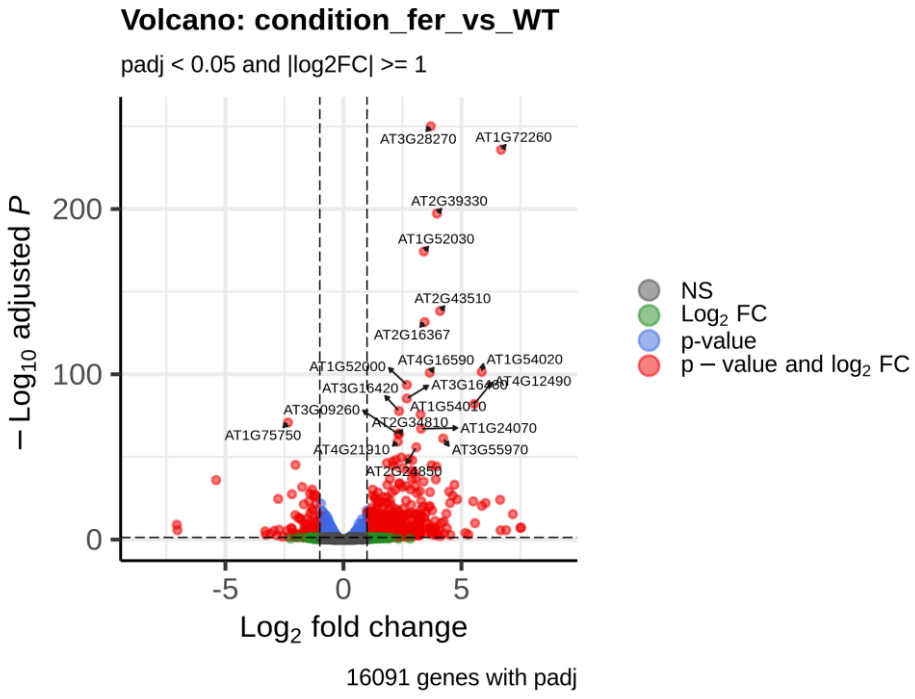



**Figure 12.** Volcano plot for fer vs WT.

Each point is a gene. Horizontal: log2 fold change (left = lower in fer, right = higher). Vertical: −log10 padj (higher = more significant). The dashed lines mark the thresholds (|log2FC| = 1 and padj = 0.05). **Red** genes pass both thresholds; blue pass only padj; green only the fold change; grey neither. The most significant genes are labelled.

## MA plot — do changes depend on expression level?

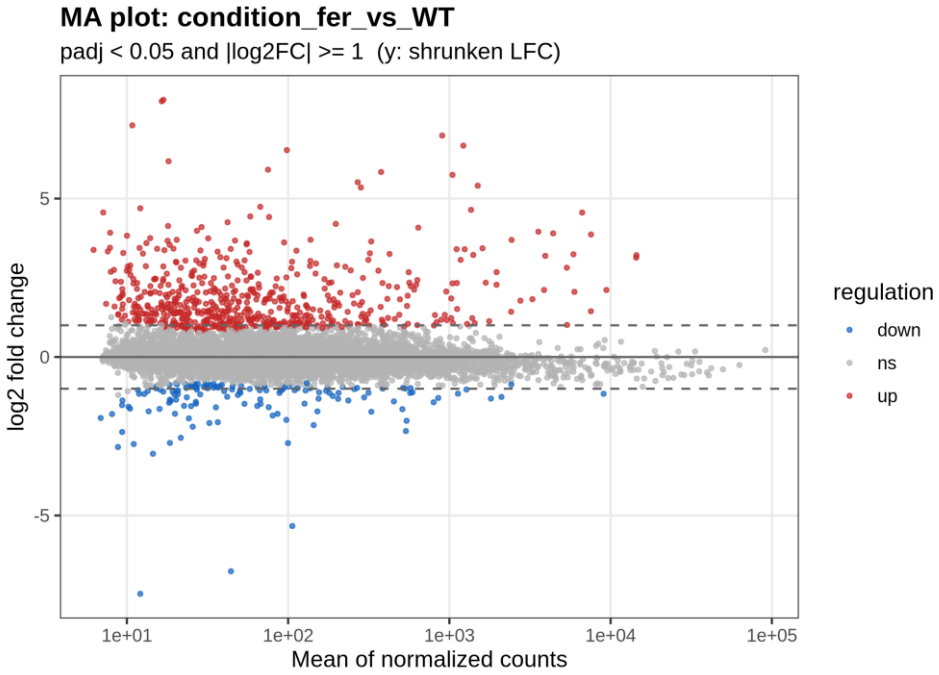



**Figure 13.** MA plot for fer vs WT (red = up, blue = down, grey = not significant).

Horizontal: average expression (baseMean, log scale). Vertical: log2 fold change (shrunken). Most genes should sit around 0. Weakly expressed genes (left) scatter more because few reads make estimates noisy; shrinkage pulls their unreliable large changes towards 0.

## Heatmap of significant genes — the pattern across samples

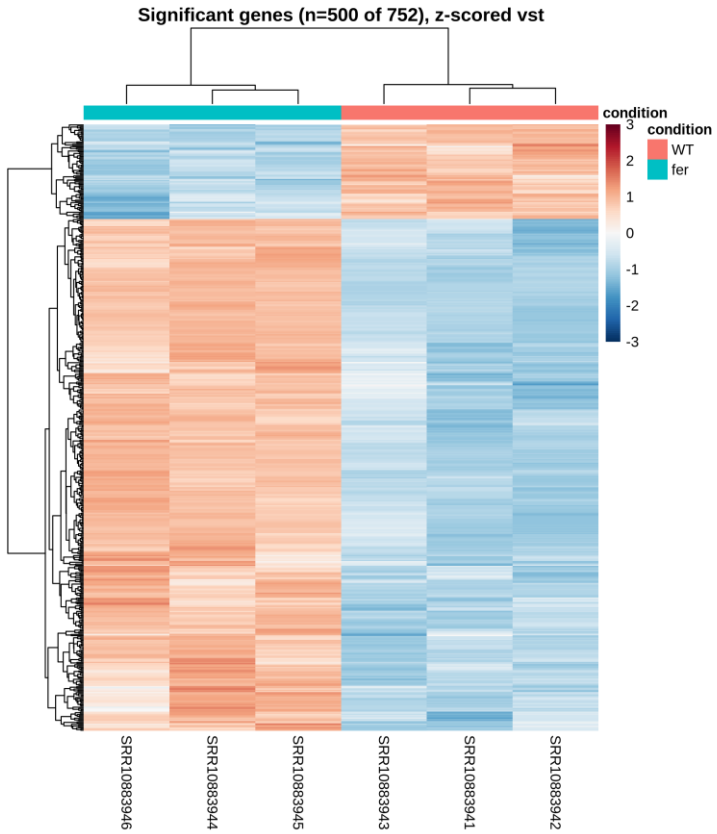



**Figure 14.** The significant genes (up to 500) in every sample; red = higher than the gene's average, blue = lower.

Rows are genes, columns are samples. Values are z-scores: for each gene, how far a sample lies above (red) or below (blue) that gene's average. A clean result shows blocks: genes high in one group and low in the other, consistent across replicates. A second heatmap shows the top genes by padj.

## Pitfalls when interpreting

- **Use padj, not pvalue.** With thousands of genes, raw p-values give many false positives.
- **Significant is not the same as important.** A tiny but very consistent change can be significant; look at the fold change and the expression level too.
- **Not significant does not mean unchanged.** It may mean too few replicates or reads to tell.
- **Check the direction.** Positive log2FC = higher in the test group than in the baseline you chose.
- **Pilot results are for testing only.** They use a fraction of the reads.
- **RNA level is not protein level.** RNA-seq measures transcripts; confirm key genes by other methods (qPCR, protein assays) when your conclusions depend on them.

# 16. A worked example: Arabidopsis FERONIA

This real analysis was run with version 1.1.0 during the pipeline's acceptance testing. It shows what a complete run looks like and how to read it.

**Table 9.** The FERONIA example in numbers.

| Item | Value |
|---|---|
| Data | public study PRJNA601158 (ENA), runs SRR10883941–SRR10883946 |
| Samples | 6: wild type (WT) × 3, *feronia* mutant (fer) × 3; single-end, 50 letters per read |
| Reads per sample | 5.6–8.9 million |
| Reference | Arabidopsis thaliana TAIR10.1 (NCBI GCF_000001735.4), 38,295 genes |
| Quality gate | TRIMMING RECOMMENDED for all samples (adapter content 7–10 %); decision: trim |
| After fastp | 92–94 % of reads kept |
| Alignment rate | 85.4–88.5 % (good) |
| Strandedness | forward (96 % of reads on the gene's strand in every sample) |
| Assigned to genes | 59–67 % (good) |
| Design | `~ condition`, baseline WT, comparison fer vs WT |
| Genes tested | 16,091 (after removing genes with too few reads) |
| Significant | **752 genes: 609 up, 143 down** (padj < 0.05 and \|log2FC\| ≥ 1) |

## How to read it

- **The quality is good:** high alignment and assignment rates, consistent across samples.
- **The groups separate cleanly** in the PCA (92.7 % of variation on PC1) and in the heatmap, with tight replicates.
- **The biology is consistent:** among the most down-regulated genes is AT3G51550, the *FERONIA* gene itself, with a log2 fold change of −5.4 (about 42× lower) in the mutant. Finding the mutated gene at the top of the list is a good sign that the samples and groups are labelled correctly.
- **Next steps** a biologist would take: look up the functions of the top genes (for Arabidopsis in TAIR), group them by pathway (for example with the optional GO enrichment), and confirm key genes by qPCR.

> **TIP**
> Doing the same sanity check in your own data is worth it: if you knocked out, silenced or over-expressed a gene, it should appear clearly in the expected direction.

# Part V · Everyday use

## 17. Resuming, changing and checking a project

### Continuing a project

Choose **Main menu → 2. Resume Existing Project** (or start with `--project` and the folder). A status table shows every step as **VALID**, **INVALID** (with the reason) or **PENDING**. Choose **Continue pipeline**: valid steps are re-checked and skipped, the rest run.

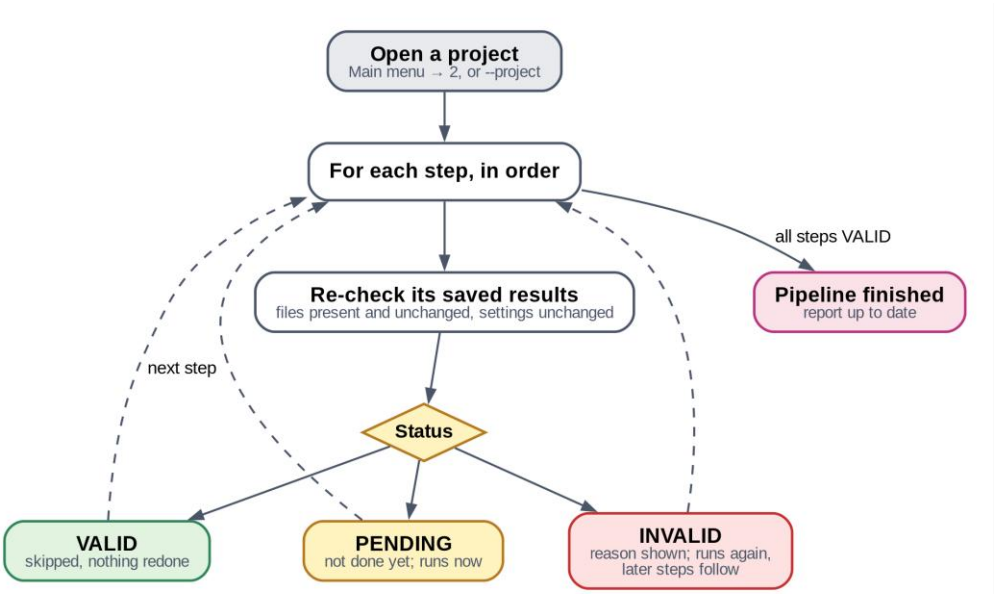



**Figure 15.** What happens when you resume.

### The project menu

| Option | Use it to… |
|---|---|
| **Continue pipeline** | run the remaining steps |
| **Re-run a specific stage** | redo one step (and everything after it) |
| **Select samples / Samples: status** | use a subset, exclude a sample, or re-include a fixed sample |
| **Choose or replace the data source** | change where the reads come from |
| **Change reference** | use another genome (alignment and later steps re-run) |
| **Experimental design** | change groups or comparisons (only DESeq2 and the report re-run) |
| **Clean up intermediate files** | free disk space safely (only files whose results are already verified) |
| **Dry run** | see what would run, without running it |

### Changing settings

**Main menu → 7. Pipeline Configuration** (or *Change settings* on a step screen) lets you change values such as the significance level (alpha, default 0.05), the fold-change threshold (default log2FC 1), trimming options or the number of threads. Every value is checked. Steps whose results depend on the changed

value become INVALID and run again when you continue; settings that cannot change results (like threads) invalidate nothing.

## Checking a finished project

**Main menu → 3. Validate Existing Project** (or `rnaseq-pipeline --validate-project FOLDER`) re-checks everything: files, counts, DESeq2 results, the report and the manifest, and that no personal credentials are stored. It ends with **PROJECT HEALTH: PASS, WARNING or FAIL** and a list of any problems. Useful before you publish, or after copying a project to another computer.

## Preparing a genome in advance

**Main menu → 5. Manage Reference Databases → Download and prepare a reference now.** No project needed. Useful to start the 1–2-hour human index the day before your data arrive. Prepared references appear as `[ready]` when you create a project.

## Freeing disk space

**Project menu → Clean up intermediate files** offers only files that are safe to delete because their results are verified: trimmed FASTQ after alignment, download caches, temporary files. Raw data, references and aligned BAM files are never offered.

# 18. When something goes wrong

Every error screen says the **step**, the **problem**, the **likely cause**, the **recommended action**, the **impact** (what was kept or discarded) and whether a **retry** is safe. Half-finished results are never used, so after fixing the cause you can simply resume.

**Table 10.** Common messages.

| You see | What it usually means | What to do |
|---|---|---|
| `rnaseq-pipeline: command not found` | the install folder is not known yet | `pipx ensurepath`, then open a new terminal |
| `HEALTH: FAIL` | a tool is missing or broken | follow the fix on the FAILED line, or Main menu → 4 |
| `fastqc: error: unrecognized arguments` | FastQC 0.13 is installed (it changed its options) | install the tested version: `mamba install -n rnaseq-tools -c conda-forge -c bioconda "fastqc=0.12.1"` |
| `HEALTH: WARNING — NCBI email not set` | no e-mail in your settings | add it (Chapter 5, step 5) |
| `SINGLE-CELL DATA DETECTED` | the dataset is single-cell RNA-seq | choose a bulk dataset; use single-cell tools |
| Download keeps failing | network or archive problem | resume later; downloads continue where they stopped |
| `truncated file` / `gzip integrity failure` | an incomplete download or copy | delete the file and download again |
| `paired read count mismatch` | R1 and R2 do not belong together | check file names and pairing; re-download both |
| `genome and annotation are not compatible` | genome and annotation from different versions | use a catalogue reference, or matching files of one release |
| `overall alignment rate … < 20%` | wrong organism or reference | check the organism in the metadata |
| Low assignment (< 30 %) | strandedness, rRNA or annotation | check strandedness and the featureCounts summary |
| `levels without biological replicates` | a group has 1 sample | add replicates or merge groups |
| `not full rank` | batch and condition confounded | remove batch from the formula |
| `R/DESeq2 step failed … SIGKILL` | R ran out of memory | close other programs or use a bigger computer, then resume |
| Step shows INVALID after resume | a file or setting changed | continue; that step runs again |
| `project is already open in another pipeline process` | two windows on one project | close the other window |
| `project filesystem is vfat` | project on a FAT32 USB disk | create the project in your home folder |

> **TIP**
>
> For more detail: `logs/pipeline_errors.log` lists only problems; the error names the tool log to open (`logs/<step>/<sample>.log`).
>
> `rnaseq-pipeline --verbose --project FOLDER` prints every command as it runs.

# 19. Frequently asked questions

**Do I need to know programming?**

No. You type a few commands to install it (Chapter 5) and then answer menus with numbers.

**Can I use my own FASTQ files?**

Yes. Choose *Local FASTQ* and point to the files or the folder. They are linked, not copied, and never changed.

**How many replicates do I need?**

At least 2 per group (required); 3 or more recommended. More replicates give more power than more reads.

**Which organisms work?**

Human, mouse and yeast from the built-in catalogue; any organism with an NCBI genome and annotation; or your own FASTA + GTF. Bacteria are supported too.

**How long does an analysis take?**

Small genomes with a pilot subset: minutes. Six human samples of 20–30 million reads on an 8-core computer: several hours, plus 1–2 hours for the first human reference.

**Can I stop and continue later?**

Yes, at any point. Use Resume; finished steps are kept.

**Can I move a project to another computer?**

Yes. Copy the whole folder; then run Validate Existing Project on the new computer.

**Can I change the significance thresholds?**

Yes: Main menu → 7. Only DESeq2 and the report re-run.

**Single-end and paired-end in one project?**

No. Make two projects.

**Can it do time courses or interactions?**

Not in this version; only group comparisons with optional batch/covariates.

**What about GO or pathway enrichment?**

Optional (Main menu → 4 → optional annotation packages). It is not part of the tested core.

**Is my NCBI key safe?**

Yes. Personal settings are never written into projects, reports, manifests or logs.

# Part VI · Reference

## 20. Tools at a glance

All tools are free, open-source and widely used in published research.

**Table 11.** The tools and their jobs.

| Tool | What it is | Its job in this pipeline | Step |
|---|---|---|---|
| **NCBI E-utilities** | NCBI's search service | search GEO and SRA for datasets; look up runs | 2 |
| **SRA Toolkit** | NCBI's download tools (prefetch, vdb-validate, fasterq-dump) | download SRA runs, check them, turn them into FASTQ | 2 |
| **curl** | a standard download program | download FASTQ files from ENA (resumable, checksum-verified) | 2 |
| **pigz** | fast compression | compress and decompress FASTQ files | 2–3 |
| **FastQC** | read quality inspector | quality profile of every FASTQ file | 4, 6 |
| **MultiQC** | report combiner | one overview of all FastQC, fastp and alignment reports | 4, 6, 9 |
| **fastp** | read cleaner | remove adapters and poor-quality ends | 6 |
| **HISAT2** | splice-aware aligner | build the genome index; place every read on the genome | 7, 8 |
| **samtools** | toolbox for aligned reads | sort, index, check and summarise BAM files; index the genome | 7–9 |
| **RSeQC** | RNA-seq quality tools | determine the library strandedness (infer_experiment) | 10 |
| **StringTie2** | transcript quantifier | estimate TPM per transcript and gene (independent view) | 11 |
| **featureCounts** | read counter (Subread package) | count reads per gene — the input of the statistics | 12 |
| **R** | statistics language | runs DESeq2 and the plots | 14–16 |
| **DESeq2** | differential expression method | normalise counts, test every gene, adjust p-values | 15 |
| **apeglm** | fold-change shrinkage | more reliable fold changes for ranking and MA plots | 15 |
| **ggplot2, pheatmap, EnhancedVolcano** | plotting packages for R | PCA, heatmaps, MA and volcano plots | 16 |
| **clusterProfiler, ReactomePA** | enrichment packages (optional) | GO and Reactome pathway enrichment | optional |
| **Miniforge, conda, mamba, Bioconda** | software installers and channels | install all tools at tested versions | setup |
| **pipx** | installer for Python programs | install the rnaseq-pipeline command | setup |

**Table 12.** Tested versions (your report lists the versions actually used).

| Tool | Tested version |
|---|---|
| FastQC | 0.12.1 (0.13 and later are not compatible yet) |
| MultiQC | 1.35 |
| fastp | 1.3.7 |
| HISAT2 | 2.2.3 |
| samtools | 1.24 |
| StringTie | 3.0.3 (2.2.3 on processors without AVX2) |
| Subread / featureCounts | 2.1.1 |
| SRA Toolkit | 3.4.1 |
| RSeQC | 5.0.5 |
| R | 4.5.3 |
| DESeq2 | 1.50.2 |

# 21. Glossary

| Term | Meaning |
|---|---|
| **Accession** | the public ID of a dataset in an archive (SRR…, GSE…, PRJNA…). |
| **Adapter** | a short synthetic DNA tag added in the lab; not part of your RNA, removed by trimming. |
| **Adjusted p-value (padj)** | a p-value corrected for testing thousands of genes; use padj < 0.05. |
| **Alignment (mapping)** | finding where in the genome each read comes from. |
| **Annotation (GTF)** | the map of the genome: where genes, transcripts and exons lie. |
| **BAM file** | the compressed file of aligned reads, sorted by genome position. |
| **Baseline (reference level)** | the group others are compared against, usually the control. |
| **baseMean** | a gene's average normalised count across all samples. |
| **Batch effect** | differences caused by processing samples on different days or runs, not by biology. |
| **Biological replicate** | an independent sample of the same group (another plant, mouse, culture). |
| **Bulk RNA-seq** | RNA-seq of a whole sample, measuring the average of many cells. |
| **cDNA** | a DNA copy of an RNA fragment, made in the lab before sequencing. |
| **Checkpoint** | the pipeline's record that a step finished and was verified. |
| **Confounding** | two factors that change together, so their effects cannot be separated. |
| **Contrast** | one comparison between two groups, e.g. treated vs control. |
| **Count matrix** | the table of reads per gene (rows) per sample (columns). |
| **Covariate** | another known factor in the design, e.g. batch or sex. |
| **Differential expression** | a statistically significant difference in a gene's expression between groups. |
| **Dispersion** | a gene's natural variability between replicates. |
| **Exon / intron** | the parts of a gene kept in mRNA (exons) and removed (introns). |
| **False discovery rate (FDR)** | the expected share of false positives among the genes called significant. |
| **FASTA** | a file format for sequences, used for genomes. |
| **FASTQ** | the file format of sequencing reads: letters plus a quality score per letter. |
| **Fold change** | the ratio between groups; log2 fold change 1 = twice as high. |
| **Gene expression** | how active a gene is, measured as its amount of mRNA. |
| **Index (genome index)** | a searchable form of the genome that makes alignment fast. |
| **Library** | the prepared, adapter-tagged fragments of one sample, ready for sequencing. |
| **Manifest** | the record of versions, settings and file fingerprints of an analysis. |
| **mRNA** | messenger RNA: the working copy of an active gene. |
| **Multi-mapping read** | a read that matches several places equally well; not counted by default. |
| **Normalisation** | making samples comparable despite different sequencing depths. |
| **Paired-end / single-end** | both ends / one end of each fragment were read. |
| **PCA** | principal component analysis: a map of samples by overall similarity. |
| **Phred quality (Q)** | a quality score: Q20 = 1 error in 100 letters, Q30 = 1 in 1,000. |
| **Pilot subset** | only the first N reads of each sample; for testing, not conclusions. |

| Term | Meaning |
|---|---|
| **p-value** | the chance of a difference at least this large if nothing really changed. |
| **Read** | one piece of sequence produced by the sequencer. |
| **Reference genome** | the organism's full DNA sequence that reads are aligned to. |
| **Sequencing depth** | the number of reads per sample. |
| **Shrinkage** | pulling uncertain fold changes towards zero to make rankings more reliable. |
| **Size factor** | a sample's normalisation factor, reflecting its sequencing depth and composition. |
| **Splice-aware aligner** | an aligner that can split a read across an intron (HISAT2). |
| **Strandedness** | whether the library keeps which DNA strand the RNA came from (forward / reverse / unstranded). |
| **TPM** | transcripts per million: abundance corrected for length and depth; for description, not testing. |
| **Trimming** | removing adapters and poor-quality ends from reads. |
| **Volcano / MA plot** | plots of all genes by fold change and significance / expression level. |
| **WSL2** | Windows Subsystem for Linux: runs Ubuntu inside Windows. |
| **z-score** | how many standard deviations a value lies above or below the gene's average; used in heatmaps. |

# 22. How to cite

When you publish results produced with this pipeline, cite **the pipeline**, **the tools it used**, and the **data** (accession numbers). Take the exact versions from the *Software versions* section of your report or from `logs/software_versions.tsv`.

## The pipeline

> **CITE AS**
>
> Vipin (2026). rnaseq-pipeline: an interactive, validated and resumable bulk RNA-seq pipeline (version 1.1.1) [Computer software]. GitHub. https://github.com/vipin839/rnaseq_pipeline

## Example methods paragraph

Adapt the parts in square brackets to your analysis:

> **METHODS (EXAMPLE)**
>
> Bulk RNA-seq data were analysed with rnaseq-pipeline v1.1.1 (Vipin, 2026). Raw reads [from accession …] were checked for integrity, and read quality was assessed with FastQC v0.12.1 (Andrews, 2010) and MultiQC v1.35 (Ewels et al., 2016). [Adapters and low-quality ends were removed with fastp v1.3.7 (Chen et al., 2018).] Reads were aligned to the [organism, assembly, annotation source and release] genome with HISAT2 v2.2.3 (Kim et al., 2019); alignments were sorted and indexed with SAMtools v1.24 (Danecek et al., 2021). Library strandedness was determined with RSeQC v5.0.5 (Wang et al., 2012) [as reverse]. Reads were counted per gene with featureCounts (Subread v2.1.1; Liao et al., 2014), and transcript abundances were estimated with StringTie v3.0.3 (Kovaka et al., 2019). Differential expression was tested with DESeq2 v1.50.2 (Love et al., 2014) in R v4.5.3, using the design [~ condition], after removing genes with fewer than 10 reads in [3] samples. P-values were adjusted with the Benjamini–Hochberg method (Benjamini & Hochberg, 1995); genes with an adjusted p-value below 0.05 and an absolute log2 fold change of at least 1 were considered differentially expressed. Fold changes for visualisation were shrunk with apeglm (Zhu et al., 2019).

## References for the tools


Andrews S. (2010). FastQC: a quality control tool for high throughput sequence data. Babraham Bioinformatics. https://www.bioinformatics.babraham.ac.uk/projects/fastqc/

Benjamini Y, Hochberg Y. (1995). Controlling the false discovery rate: a practical and powerful approach to multiple testing. Journal of the Royal Statistical Society, Series B, 57(1), 289–300. https://doi.org/10.1111/j.2517-6161.1995.tb02031.x

Blighe K, Rana S, Lewis M. EnhancedVolcano: publication-ready volcano plots with enhanced colouring and labeling. R/Bioconductor package. https://github.com/kevinblighe/EnhancedVolcano

Chen S, Zhou Y, Chen Y, Gu J. (2018). fastp: an ultra-fast all-in-one FASTQ preprocessor. Bioinformatics, 34(17), i884–i890. https://doi.org/10.1093/bioinformatics/bty560

Danecek P, Bonfield JK, Liddle J, et al. (2021). Twelve years of SAMtools and BCFtools. GigaScience, 10(2), giab008. https://doi.org/10.1093/gigascience/giab008

Ewels P, Magnusson M, Lundin S, Käller M. (2016). MultiQC: summarize analysis results for multiple tools and samples in a single report. Bioinformatics, 32(19), 3047–3048. https://doi.org/10.1093/bioinformatics/btw354

Grüning B, Dale R, Sjödin A, et al. (2018). Bioconda: sustainable and comprehensive software distribution for the life sciences. Nature Methods, 15(7), 475–476. https://doi.org/10.1038/s41592-018-0046-7

Kim D, Paggi JM, Park C, Bennett C, Salzberg SL. (2019). Graph-based genome alignment and genotyping with HISAT2 and HISAT-genotype. Nature Biotechnology, 37(8), 907–915. https://doi.org/10.1038/s41587-019-0201-4

Kolde R. pheatmap: Pretty Heatmaps. R package. https://CRAN.R-project.org/package=pheatmap

Kovaka S, Zimin AV, Pertea GM, Razaghi R, Salzberg SL, Pertea M. (2019). Transcriptome assembly from long-read RNA-seq alignments with StringTie2. Genome Biology, 20, 278. https://doi.org/10.1186/s13059-019-1910-1

Leinonen R, Sugawara H, Shumway M. (2011). The Sequence Read Archive. Nucleic Acids Research, 39(Database issue), D19–D21. https://doi.org/10.1093/nar/gkq1019

Liao Y, Smyth GK, Shi W. (2014). featureCounts: an efficient general purpose program for assigning sequence reads to genomic features. Bioinformatics, 30(7), 923–930. https://doi.org/10.1093/bioinformatics/btt656

Love MI, Huber W, Anders S. (2014). Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. Genome Biology, 15, 550. https://doi.org/10.1186/s13059-014-0550-8

R Core Team. R: A language and environment for statistical computing. R Foundation for Statistical Computing, Vienna, Austria. https://www.R-project.org/

Wang L, Wang S, Li W. (2012). RSeQC: quality control of RNA-seq experiments. Bioinformatics, 28(16), 2184–2185. https://doi.org/10.1093/bioinformatics/bts356

Wickham H. (2016). ggplot2: Elegant Graphics for Data Analysis. Springer-Verlag, New York.

Zhu A, Ibrahim JG, Love MI. (2019). Heavy-tailed prior distributions for sequence count data: removing the noise and preserving large differences. Bioinformatics, 35(12), 2084–2092. https://doi.org/10.1093/bioinformatics/bty895

If you used the optional enrichment: Wu T, Hu E, Xu S, et al. (2021). clusterProfiler 4.0: a universal enrichment tool for interpreting omics data. The Innovation, 2(3), 100141. https://doi.org/10.1016/j.xinn.2021.100141 · Yu G, He QY. (2016). ReactomePA: an R/Bioconductor package for reactome pathway analysis and visualization. Molecular BioSystems, 12(2), 477–479. https://doi.org/10.1039/C5MB00663E

> **TIP**
>
> Also cite the data: the accession numbers (GEO, SRA, ENA, BioProject) of every dataset you used, and the source and release of the reference genome and annotation (shown in the report).

# 23. Getting help and licence

- **Source code and documentation:** https://github.com/vipin839/rnaseq_pipeline
- **Report a problem or ask a question:** https://github.com/vipin839/rnaseq_pipeline/issues — describe what you did, copy the error screen, and attach `logs/pipeline_errors.log` (it never contains your NCBI key).
- **Technical documentation** (for advanced users) is in the `docs/` folder of the download: installation, methods with every parameter, troubleshooting, testing and the developer guide.
- **Licence:** MIT. You may use, copy, change and share the software freely, including for commercial work, provided the copyright notice is kept. It comes without warranty.

# Appendix · One-page cheat sheet

## Install (once)

```
cd ~
curl -L -O https://github.com/conda-
forge/miniforge/releases/latest/download/Miniforge3-Linux-x86_64.sh
bash Miniforge3-Linux-x86_64.sh -b -p ~/miniforge3
git clone https://github.com/vipin839/rnaseq_pipeline.git
sudo apt install -y pipx && pipx ensurepath        (then reopen the terminal)
pipx install ~/rnaseq_pipeline
~/miniforge3/bin/mamba env create -f "$(rnaseq-pipeline --runtime-env tools)"
~/miniforge3/bin/mamba env create -f "$(rnaseq-pipeline --runtime-env r)"
rnaseq-pipeline --check
```

## Use

| To… | Type / choose |
|---|---|
| start the program | `rnaseq-pipeline` |
| start a new analysis | Main menu → 1 |
| continue an analysis | Main menu → 2, or `rnaseq-pipeline --project FOLDER` |
| check a project | Main menu → 3, or `rnaseq-pipeline --validate-project FOLDER` |
| repair or install tools | Main menu → 4 |
| prepare a genome in advance | Main menu → 5 |
| open an earlier report | Main menu → 6 |
| change thresholds or settings | Main menu → 7 |
| check the installation | `rnaseq-pipeline --check` |

## Read the results

| Look at | Good sign |
|---|---|
| `reports/final_pipeline_report.html` | no errors; quality numbers in the good range (Chapter 15) |
| PCA plot | replicates together, groups apart |
| `upregulated.tsv`, `downregulated.tsv` | genes with padj < 0.05 and \|log2FC\| ≥ 1 |
| Volcano and MA plots | red/blue genes on both sides; most genes around 0 |
| Your known genes | a knocked-out or over-expressed gene changes in the expected direction |

# Development and credits

rnaseq-pipeline was designed, developed and is maintained by

# Vipin


| | |
|---|---|
| **Website** | https://www.vipin.social/ |
| **E-mail** | sainivipin839@gmail.com |
| **GitHub** | github.com/vipin839/ |
| **Repository** | github.com/vipin839/rnaseq_pipeline |
| **Issues and questions** | github.com/vipin839/rnaseq_pipeline/issues |


## About the project

Goal: a bulk RNA-seq analysis that biologists can run, understand and trust without programming.

**Table 13.** Version history.

| Version | Date | Highlights |
|---|---|---|
| 1.0.0 | 19 September 2026 | first complete pipeline: download to DESeq2, checkpoints, HTML report |
| 1.1.0 | 25 September 2026 | independent re-checks of every result, installable command, real-data acceptance |
| 1.1.1 | 19 October 2026 | FastQC kept at the tested 0.12 line; this illustrated user guide |

## Feedback, collaboration and contributions

Questions, bug reports and suggestions are welcome: open an issue on GitHub or write by e-mail. For collaborations, training or analysis support, use the website or e-mail above. With a problem report, include the error screen and `logs/pipeline_errors.log` (it never contains personal credentials).

## Acknowledgements


This pipeline stands on the work of the open-source bioinformatics community: the developers of FastQC, MultiQC, fastp, HISAT2, SAMtools, RSeQC, StringTie2, Subread/featureCounts, the SRA Toolkit, R, Bioconductor and DESeq2, and the conda-forge and Bioconda projects that distribute them. Public data and references come from NCBI, ENA, Ensembl, GENCODE and UCSC; the worked example uses dataset PRJNA601158.


## Licence and disclaimer


Copyright © 2026 Vipin. rnaseq-pipeline is free software released under the MIT licence. It is provided "as is", without warranty of any kind. Results should be interpreted by the user in the context of the experiment, and key findings confirmed by independent methods. When you use the pipeline in published work, please cite it as described in Chapter 22.